

Meetbowl_CI/CD_계획서

1. 시스템 아키텍처

Meetbowl은 프론트엔드, 백엔드 API 서버, AI 서버, STT 서버, 인프라 레포를 분리하여 관리한다. 각 애플리케이션 레포는 GitHub Actions를 통해 테스트, 빌드, 배포를 자동화하고, EC2 내부의 infra 레포가 실제 컨테이너 실행 구성을 담당한다.

구분	기술/서비스	배포 위치	역할
Frontend	Vite, S3, CloudFront, Route53	S3 + CloudFront	정적 웹 애플리케이션 빌드 및 배포
Backend Server	Spring Boot, Docker, ECR	EC2 Docker Compose	API, 인증/인가, 회의/메일/회의록 도메인 처리
AI Server	FastAPI, uv, Docker, ECR	EC2 Docker Compose	회의록 생성, LLM 호출, AI 요약 처리
STT Server	Node.js, Docker, ECR	EC2 Docker Compose	LiveKit 오디오 기반 실시간 STT 중계
Infra Repository	Docker Compose, Nginx, 배포 스크립트	EC2 내부 Git 레포	서비스별 컨테이너 실행 및 운영 설정 관리
Common Infra	MariaDB, Redis, RabbitMQ, Qdrant, LiveKit	EC2 Docker Compose	공통 데이터 저장, 메시징, 벡터 검색, 실시간 미디어 처리

2. 전체 CI/CD 흐름

1. 개발자가 각 서비스 레포지토리의 main 브랜치로 Pull Request를 생성한다.
2. Pull Request 이벤트에서는 테스트와 빌드 검증만 수행하며 운영 배포는 수행하지 않는다.
3. main 브랜치에 merge 또는 push되면 테스트 이후 Docker 이미지 빌드가 수행된다.
4. 빌드된 이미지는 AWS ECR에 latest 태그와 GitHub SHA 태그로 업로드된다.
5. GitHub Actions가 EC2에 SSH로 접속한다.
6. EC2 내부 infra 레포의 서비스별 배포 스크립트를 실행한다.
7. 배포 스크립트는 ECR에서 지정된 SHA 이미지를 pull한다.
8. Docker Compose를 통해 해당 서비스 컨테이너를 교체한다.

9. Frontend는 별도 흐름으로 dist 결과물을 S3에 업로드하고 CloudFront 캐시 무효화를 수행한다.

핵심 요약: Meetbowl의 서버 배포는 GitHub Actions에서 테스트와 Docker 이미지 빌드를 수행하고, AWS ECR에 이미지를 저장한 뒤, EC2 내부 infra 레포의 배포 스크립트를 실행하여 Docker Compose 기반으로 서비스를 교체하는 구조이다. Frontend는 S3와 CloudFront를 이용해 정적 파일 배포를 수행한다.

3. CI/CD 실행 조건 및 공통 정책

이벤트	실행 작업	운영 배포 여부
pull_request → main	테스트 및 빌드 검증	배포하지 않음
push → main	테스트, 이미지 빌드, ECR push, EC2 배포	배포함
workflow_dispatch	수동 실행	배포함

- concurrency를 사용하여 같은 ref에서 중복 실행되는 워크플로를 제한한다.
- 이미지 태그는 latest와 github.sha를 함께 사용한다.
- Pull Request에서는 운영 Secret을 사용하지 않고 테스트 전용 환경변수를 주입한다.
- AWS 인증은 장기 Access Key 대신 GitHub Actions OIDC 기반 IAM Role Assume 방식을 사용한다.

4. Frontend CI/CD

Frontend는 Vite 기반 정적 웹 애플리케이션이며, 빌드 결과물인 dist 디렉토리를 S3에 업로드하고 CloudFront를 통해 배포한다.

항목	내용
런타임	Node.js 22
패키지 설치	npm ci
테스트	npm test
빌드	npm run build
배포 대상	S3 Bucket
캐시 갱신	CloudFront Invalidation

4.1 배포 흐름

1. GitHub Actions에서 checkout을 수행한다.

2. Node.js 22 환경을 구성하고 npm 캐시를 사용한다.
3. npm ci로 의존성을 설치한다.
4. npm test로 테스트를 수행한다.
5. VITE_API_BASE_URL 환경변수를 주입하고 npm run build를 수행한다.
6. Pull Request가 아닌 경우 dist 결과물을 artifact로 업로드한다.
7. deploy job에서 artifact를 다운로드한다.
8. AWS OIDC 인증 후 S3에 파일을 업로드한다.
9. dist/assets는 장기 캐시, 그 외 파일은 no-cache 정책으로 업로드한다.
10. CloudFront create-invalidation을 수행하여 최신 파일을 반영한다.

4.2 배포 전략

- 정적 자산은 S3에 업로드하고 CloudFront를 통해 사용자에게 제공한다.
- assets 디렉토리는 파일명 해시를 전제로 public,max-age = 31536000,immutable 캐시 정책을 적용한다.
- index.html 등 애플리케이션 진입 파일은 no-cache 정책으로 업로드한다.
- 배포 후 CloudFront 캐시 무효화를 수행하여 최신 버전을 즉시 제공한다.
- 롤백은 이전 커밋 재배포 또는 S3 버전 관리를 통해 수행할 수 있다.

4.3 파이프라인 코드

```
name: Frontend Deploy

on:
  pull_request:
    branches:
      - main
  push:
    branches:
      - main
  workflow_dispatch:

concurrency:
  group: frontend-production-${{ github.ref }}
```

```

cancel-in-progress: ${{{ github.event_name == 'pull_request' }}}

jobs:
  build:
    name: Test and Build
    runs-on: ubuntu-latest
    timeout-minutes: 15

    steps:
      - name: Checkout
        uses: actions/checkout@v5

      - name: Set up Node.js
        uses: actions/setup-node@v5
        with:
          node-version: "22"
          cache: npm

      - name: Install dependencies
        run: npm ci

      - name: Run tests
        run: npm test

      - name: Build
        env:
          VITE_API_BASE_URL: ${{{ secrets.VITE_API_BASE_URL || '/api/v1' }}}
        run: npm run build

      - name: Upload build artifact
        if: github.event_name != 'pull_request'
        uses: actions/upload-artifact@v5
        with:
          name: frontend-dist-${{{ github.sha }}}
          path: dist/
          if-no-files-found: error

```

```

        retention-days: 1

deploy:
  name: Deploy to S3 and CloudFront
  if: github.event_name != 'pull_request'
  needs: build
  runs-on: ubuntu-latest
  timeout-minutes: 15
  permissions:
    contents: read
    id-token: write

  steps:
    - name: Validate deployment configuration
      env:
        AWS_ROLE_ARN: ${ secrets.AWS_ROLE_ARN }
        AWS_REGION: ${ secrets.AWS_REGION }
        S3_BUCKET: ${ secrets.S3_BUCKET }
        CLOUDFRONT_DISTRIBUTION_ID: ${ secrets.CLOUDFRONT_DISTRIBUTION_ID }
      run: |
        set -euo pipefail

        for variable_name in \
          AWS_ROLE_ARN \
          AWS_REGION \
          S3_BUCKET \
          CLOUDFRONT_DISTRIBUTION_ID
        do
          if [ -z "${!variable_name}" ]; then
            echo "::error::Missing GitHub production environment variable: ${variable_name}"
            exit 1
          fi
        done

    - name: Download build artifact
      uses: actions/download-artifact@v5

```

```

with:
  name: frontend-dist-${{ github.sha }}
  path: dist

- name: Configure AWS credentials
  uses: aws-actions/configure-aws-credentials@v5
  with:
    role-to-assume: ${ secrets.AWS_ROLE_ARN }
    aws-region: ${ secrets.AWS_REGION }

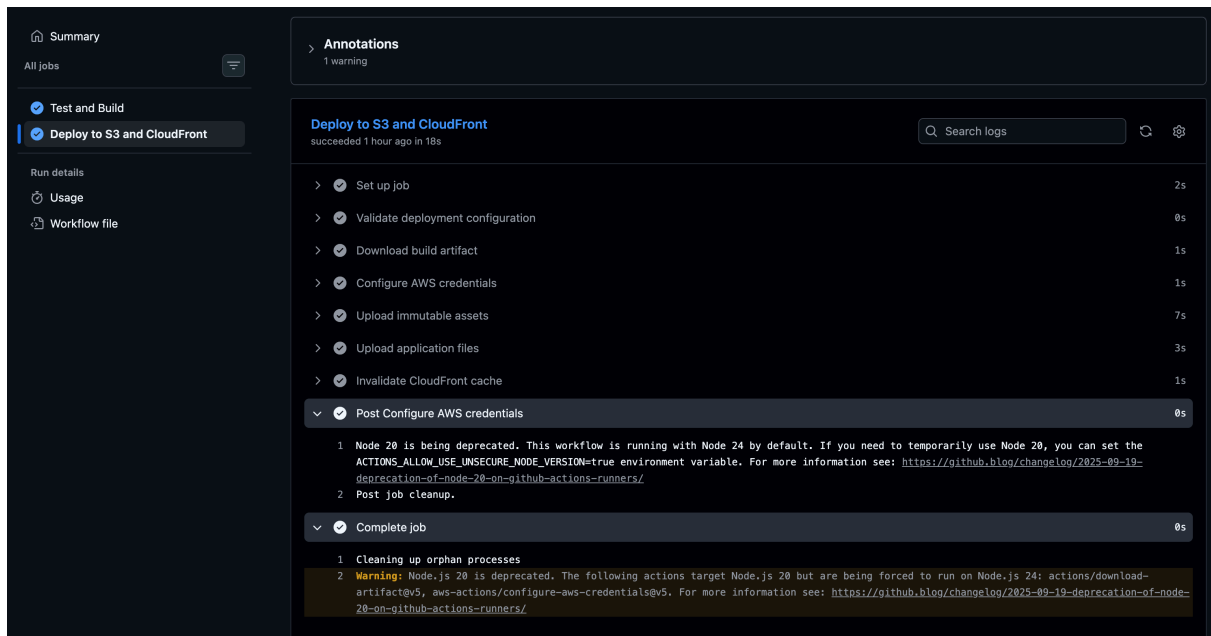
- name: Upload immutable assets
  env:
    S3_BUCKET: ${ secrets.S3_BUCKET }
  run: |
    aws s3 sync dist/assets "s3://${S3_BUCKET}/asset
s" \
    --delete \
    --cache-control "public,max-age=31536000,immuta
ble"

- name: Upload application files
  env:
    S3_BUCKET: ${ secrets.S3_BUCKET }
  run: |
    aws s3 sync dist "s3://${S3_BUCKET}" \
    --delete \
    --exclude "assets/*" \
    --cache-control "no-cache"

- name: Invalidate CloudFront cache
  env:
    CLOUDFRONT_DISTRIBUTION_ID: ${ secrets.CLOUDFRONT
T_DISTRIBUTION_ID }
  run: |
    aws cloudfront create-invalidation \
    --distribution-id "${CLOUDFRONT_DISTRIBUTION_I
D}" \
    --paths "/*"

```

4.3 배포 결과



5. Backend Server CI/CD

Backend Server는 Spring Boot 기반 API 서버이며, Docker 이미지로 패키징하여 AWS ECR에 업로드한 뒤 EC2에서 Docker Compose로 실행한다.

항목	내용
런타임	Java 25
빌드 도구	Gradle
테스트 명령어	./gradlew test --no-daemon
테스트 의존 서비스	MariaDB 11.8, Redis 7.4, RabbitMQ 3.13-management
이미지 저장소	AWS ECR - meetbowl-be
배포 방식	EC2 SSH 접속 후 scripts/deploy-be.sh 실행

5.1 CI 흐름

1. Pull Request 또는 main push 발생 시 GitHub Actions가 실행된다.
2. MariaDB, Redis, RabbitMQ 서비스 컨테이너를 GitHub Actions 내부에서 실행한다.
3. Java 25와 Gradle 환경을 구성한다.
4. gradlew 실행 권한을 부여한다.
5. test profile 및 테스트용 환경변수를 주입한다.

6. ./gradlew test -no-daemon 명령어로 백엔드 테스트를 수행한다.

5.2 테스트 환경

서비스	이미지	용도
MariaDB	mariadb:11.8	JPA/Repository/통합 테스트용 DB
Redis	redis:7.4-alpine	캐시 및 실시간 상태 처리 테스트
RabbitMQ	rabbitmq:3.13-management	이벤트 발행/소비 테스트

- 운영 DB, Redis, RabbitMQ에 의존하지 않고 GitHub Actions 내부 서비스 컨테이너로 테스트한다.
- JWT, Internal Token, LiveKit, STT, AI, S3 관련 값은 테스트용 환경변수로 주입한다.
- AWS 자격증명도 테스트용 dummy 값을 사용하여 운영 Secret 사용을 방지한다.

5.3 CD 흐름

1. 테스트가 성공하면 Docker 이미지 빌드 job이 실행된다.
2. GitHub Actions OIDC로 AWS IAM Role을 Assume한다.
3. ECR 에 로그인한다.
4. Docker Buildx로 이미지를 빌드한다.
5. ECR에 latest 태그와 GitHub SHA 태그로 이미지를 push한다.
6. deploy job에서 EC2 에 SSH 로 접속한다.
7. SSM Parameter Store에서 CORS Origin 설정을 조회하여 배포 환경변수로 전달한다.
8. EC2 내부 infra 레포의 scripts/deploy-be.sh를 실행한다.

5.4 파이프라인 코드

```
name: Meetbowl BE CI/CD
```

```
on:
```

```
  pull_request:
```

```
    branches:
```

```
      - main
```

```
  push:
```

```

    branches:
      - main
  workflow_dispatch:

  concurrency:
    group: meetbowl-be-${{ github.ref }}
    cancel-in-progress: true

  env:
    JAVA_VERSION: "25"
    AWS_REGION: ap-southeast-2
    ECR_REPOSITORY: meetbowl-be
    IMAGE_TAG: ${{ github.sha }}

  jobs:
    test:
      name: Test
      runs-on: ubuntu-latest

      services:
        mariadb:
          image: mariadb:11.8
          env:
            MARIADB_ROOT_PASSWORD: root-password
            MARIADB_DATABASE: meetbowl
            MARIADB_USER: meetbowl
            MARIADB_PASSWORD: test-meetbowl-password
          ports:
            - 3306:3306
          options: >-
            --health-cmd="mariadb-admin ping -h 127.0.0.1 -u
root -proot-password --silent"
            --health-interval=10s
            --health-timeout=5s
            --health-retries=20

        redis:
          image: redis:7.4-alpine

```

```

ports:
  - 6379:6379
options: >-
  --health-cmd="redis-cli ping"
  --health-interval=10s
  --health-timeout=5s
  --health-retries=20

rabbitmq:
  image: rabbitmq:3.13-management
  env:
    RABBITMQ_DEFAULT_USER: meetbowl
    RABBITMQ_DEFAULT_PASS: test-rabbitmq-password
    RABBITMQ_DEFAULT_VHOST: /
  ports:
    - 5672:5672
    - 15672:15672
  options: >-
    --health-cmd="rabbitmq-diagnostics -q ping"
    --health-interval=10s
    --health-timeout=5s
    --health-retries=20

steps:
  - name: Checkout pull request head
    if: github.event_name == 'pull_request'
    uses: actions/checkout@v4
    with:
      ref: ${ github.event.pull_request.head.sha }

  - name: Checkout push/workflow ref
    if: github.event_name != 'pull_request'
    uses: actions/checkout@v4

  - name: Set up Java
    uses: actions/setup-java@v4
    with:
      distribution: temurin

```

```

    java-version: ${{ env.JAVA_VERSION }}

- name: Set up Gradle
  uses: gradle/actions/setup-gradle@v4

- name: Grant execute permission for gradlew
  run: chmod +x ./gradlew

- name: Show checked out revision
  run: |
    echo "event_name=${{ github.event_name }}"
    echo "github_sha=${{ github.sha }}"
    echo "pr_head_sha=${{ github.event.pull_request.h
ead.sha }}"
    git rev-parse HEAD

- name: Run backend tests
  env:
    SPRING_PROFILES_ACTIVE: test
    MEETBOWL_DB_URL: jdbc:mariadb://127.0.0.1:3306/me
etbowl

    MEETBOWL_DB_USERNAME: meetbowl
    MEETBOWL_DB_PASSWORD: test-meetbowl-password
    MEETBOWL_REDIS_HOST: 127.0.0.1
    MEETBOWL_REDIS_PORT: 6379
    MEETBOWL_RABBITMQ_HOST: 127.0.0.1
    MEETBOWL_RABBITMQ_PORT: 5672
    MEETBOWL_RABBITMQ_USERNAME: meetbowl
    MEETBOWL_RABBITMQ_PASSWORD: test-rabbitmq-passwor
d

    MEETBOWL_RABBITMQ_VHOST: /
    MEETBOWL_RABBITMQ_EXCHANGE: meetbowl.topic
    MEETBOWL_RABBITMQ_MINUTES_GENERATED_QUEUE: api.mi
nutes.generated

    MEETBOWL_JWT_SECRET: meetbowl-github-actions-jwt-
secret-32bytes
    MEETBOWL_INTERNAL_TOKEN: meetbowl-github-actions-
internal-token-32bytes

```

```

    MEETBOWL_LIVEKIT_URL: http://127.0.0.1:7880
    MEETBOWL_LIVEKIT_API_KEY: devkey
    MEETBOWL_LIVEKIT_API_SECRET: local-livekit-secret
- change-me-123456
    MEETBOWL_STT_BASE_URL: http://127.0.0.1:3000/api/
v1
    MEETBOWL_AI_BASE_URL: http://127.0.0.1:8000
    MEETBOWL_ELASTICSEARCH_URL: http://127.0.0.1:9200
    AWS_REGION: ap-northeast-2
    AWS_ACCESS_KEY_ID: test
    AWS_SECRET_ACCESS_KEY: test
    S3_BUCKET: meetbowl-files
    S3_ENDPOINT: http://127.0.0.1:4566
run: ./gradlew test --no-daemon

```

build-and-push:

```

name: Build And Push Image
runs-on: ubuntu-latest
needs:
  - test
if: github.event_name != 'pull_request'
permissions:
  id-token: write
  contents: read

```

steps:

```

- name: Checkout
  uses: actions/checkout@v4

- name: Configure AWS credentials
  uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: ${ secrets.AWS_GITHUB_ACTIONS_ROLE_ARN }
    aws-region: ${ env.AWS_REGION }

- name: Log in to ECR
  id: login-ecr

```

```

    uses: aws-actions/amazon-ecr-login@v2

- name: Set up Docker Buildx
  uses: docker/setup-buildx-action@v3

- name: Build and push image
  uses: docker/build-push-action@v6
  with:
    context: .
    file: ./Dockerfile
    push: true
    tags: |
      ${ steps.login-ecr.outputs.registry }/${ env.ECR_REPOSITORY }:latest
      ${ steps.login-ecr.outputs.registry }/${ env.ECR_REPOSITORY }:${ env.IMAGE_TAG }
    cache-from: type=gha
    cache-to: type=gha,mode=max

deploy:
  name: Deploy
  runs-on: ubuntu-latest
  needs:
    - build-and-push
  if: github.event_name != 'pull_request'
  permissions:
    id-token: write
    contents: read

  steps:
    - name: Configure AWS credentials
      uses: aws-actions/configure-aws-credentials@v4
      with:
        role-to-assume: ${ secrets.AWS_GITHUB_ACTIONS_ROLE_ARN }
        aws-region: ${ env.AWS_REGION }

    - name: Load CORS origin patterns from SSM

```

```

run: |
    set -euo pipefail

    MEETBOWL_CORS_ALLOWED_ORIGIN_PATTERNS_SSM_NAME="/
meetbowl/prod/be/MEETBOWL_CORS_ALLOWED_ORIGIN_PATTERNS"

    MEETBOWL_CORS_ALLOWED_ORIGIN_PATTERNS="$(
        aws ssm get-parameter \
            --name "$MEETBOWL_CORS_ALLOWED_ORIGIN_PATTERN
S_SSM_NAME" \
            --with-decryption \
            --query 'Parameter.Value' \
            --output text
    )"

    echo "::add-mask::$MEETBOWL_CORS_ALLOWED_ORIGIN_P
ATTERNS"

    echo "MEETBOWL_CORS_ALLOWED_ORIGIN_PATTERNS=$MEET
BOWL_CORS_ALLOWED_ORIGIN_PATTERNS" >> "$GITHUB_ENV"

- name: Deploy on EC2
  env:
    SSH_HOST: ${ secrets.MEETBOWL_EC2_HOST }
    SSH_USER: ${ secrets.MEETBOWL_EC2_USER }
    SSH_KEY: ${ secrets.MEETBOWL_EC2_SSH_KEY }
    DEPLOY_PATH: ${ secrets.MEETBOWL_DEPLOY_PATH }
    AWS_REGION: ${ env.AWS_REGION }
    ECR_REPOSITORY: ${ env.ECR_REPOSITORY }
    IMAGE_TAG: ${ env.IMAGE_TAG }
    MEETBOWL_CORS_ALLOWED_ORIGIN_PATTERNS: ${ env.ME
ETBOWL_CORS_ALLOWED_ORIGIN_PATTERNS }
  run: |
    set -euo pipefail

    mkdir -p ~/.ssh
    printf '%s\n' "$SSH_KEY" > ~/.ssh/deploy_key
    chmod 600 ~/.ssh/deploy_key
    ssh-keyscan "$SSH_HOST" >> ~/.ssh/known_hosts

```

```
ssh -i ~/.ssh/deploy_key "$SSH_USER@$SSH_HOST" \
    "DEPLOY_PATH='$DEPLOY_PATH' AWS_REGION='$AWS_REGION' ECR_REPOSITORY='$ECR_REPOSITORY' IMAGE_TAG='$IMAGE_TAG' MEETBOWL_CORS_ALLOWED_ORIGIN_PATTERNS='$MEETBOWL_CORS_ALLOWED_ORIGIN_PATTERNS' bash -s" <<'EOF'
set -euo pipefail

cd "$DEPLOY_PATH"

if [ ! -x ./scripts/deploy-be.sh ]; then
    echo "expected deploy script at $DEPLOY_PATH/scripts/deploy-be.sh"
    exit 1
fi

./scripts/deploy-be.sh
EOF
```

5.5 배포 결과

The screenshot shows a GitHub Actions workflow run for the 'Deploy' job, which succeeded 5 minutes ago in 36s. The workflow is titled 'Deploy on EC2' and shows a list of steps:

- Test** (succeeded)
- Build And Push Image** (succeeded)
- Deploy** (succeeded)

The 'Deploy' step details show the following output:

```
74 d9c534cc3184 Extracting 18
75 d9c534cc3184 Extracting 18
76 d9c534cc3184 Extracting 18
77 d9c534cc3184 Extracting 18
78 d9c534cc3184 Pull complete 0B
79 Image 476663692671.dkr.ecr.ap-southeast-2.amazonaws.com/meetbowl-be:046be2f60f5677d14398d0f9c00e2dc5770292aa Pulled
80 time="2026-06-30T06:56:38Z" level=warning msg="Found orphan containers ([meetbowl-stt meetbowl-ai]) for this project. If you removed or
renamed this service in your compose file, you can run this command with the --remove-orphans flag to clean it up."
81 Container meetbowl-redis Running
82 Container meetbowl-rabbitmq Running
83 Container meetbowl-be Recreate
84 Container meetbowl-livekit Running
85 Container meetbowl-nginx Running
86 Container meetbowl-qdrant Running
87 Container meetbowl-elasticsearch Running
88 Container meetbowl-be Recreated
89 Container meetbowl-redis Waiting
90 Container meetbowl-rabbitmq Waiting
91 Container meetbowl-rabbitmq Healthy
92 Container meetbowl-redis Healthy
93 Container meetbowl-be Starting
94 Container meetbowl-be Started
95 nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
96 nginx: configuration file /etc/nginx/nginx.conf test is successful
97 Notice: 2026/06/30 06:56:51 [notice] 143#143: signal process started
98 curl: (55) Failed sending HTTP request
```

Below the 'Deploy' step, there is a 'Post Configure AWS credentials' step (succeeded) and a 'Complete job' step (succeeded).

At the bottom, a warning message states: "Warning: Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: aws-actions/configure-aws-credentials@v4. For more information see: <https://github.blog/changelog/2025-09-19-deprecation-of-node-20-on-github-actions-runners/>".

6. AI Server CI/CD

AI Server는 FastAPI 기반 서버이며, 회의록 생성, LLM 호출, AI 기반 요약 작업을 담당한다. Python 의존성 관리는 uv를 사용한다.

항목	내용
런타임	Python 3.12
패키지 관리	uv
의존성 설치	uv sync -frozen -dev
테스트 명령어	uv run pytest -q
이미지 저장소	AWS ECR - meetbowl-ai
배포 방식	EC2 SSH 접속 후 scripts/deploy-ai.sh 실행

6.1 CI 흐름

1. Pull Request 또는 main push 발생 시 GitHub Actions가 실행된다.
2. Python 3.12 환경을 구성한다.
3. astral-sh/setup-uv를 사용하여 uv를 설치하고 캐시를 활성화한다.
4. uv sync -frozen -dev 명령어로 lock 파일 기준 의존성을 설치한다.
5. 테스트용 INTERNAL_TOKEN, BE_BASE_URL, GEMINI_API_KEY, OPENAI_API_KEY를 주입한다.
6. uv run pytest -q 명령어로 테스트를 수행한다.

6.2 CD 흐름

1. test job이 성공하면 build-and-push job이 실행된다.
2. GitHub Actions OIDC로 AWS 인증을 수행한다.
3. ECR 에 로그인한다.
4. Docker Buildx를 설정한다.
5. Docker 이미지를 빌드한다.
6. ECR에 latest 태그와 GitHub SHA 태그로 push한다.
7. deploy job에서 EC 2 에 SSH 로 접속한다.
8. EC2 내부 infra 레포의 scripts/deploy-ai.sh를 실행한다.

6.3 배포 전략

- AI Server는 BE Server와 동일하게 Docker 이미지 기반 배포 전략을 사용한다.

- EC2에서는 코드를 직접 빌드하지 않고, GitHub Actions에서 빌드된 이미지를 pull하여 실행한다.
- LLM API Key는 운영 환경에서 Secret 또는 Parameter Store로 관리하고, CI 테스트에서는 dummy key를 사용한다.

6.4 파이프라인 코드

```
name: Meetbowl AI CI/CD

on:
  pull_request:
    branches:
      - main
  push:
    branches:
      - main
  workflow_dispatch:

concurrency:
  group: meetbowl-ai-${{ github.ref }}
  cancel-in-progress: true

env:
  PYTHON_VERSION: "3.12"
  AWS_REGION: ap-southeast-2
  ECR_REPOSITORY: meetbowl-ai
  IMAGE_TAG: "${{ github.sha }}"

jobs:
  test:
    name: Test
    runs-on: ubuntu-latest

    steps:
      - name: Checkout pull request head
        if: github.event_name == 'pull_request'
        uses: actions/checkout@v4
        with:
```

```

    ref: ${GITHUB_EVENT_PULL_REQUEST_HEAD_SHA}

- name: Checkout push/workflow ref
  if: github.event_name != 'pull_request'
  uses: actions/checkout@v4

- name: Set up Python
  uses: actions/setup-python@v5
  with:
    python-version: ${PYTHON_VERSION}

- name: Set up uv
  uses: astral-sh/setup-uv@v6
  with:
    enable-cache: true

- name: Sync dependencies
  run: uv sync --frozen --dev

- name: Run tests
  env:
    INTERNAL_TOKEN: meetbowl-github-actions-internal-token-32bytes
    BE_BASE_URL: http://127.0.0.1:8080
    GEMINI_API_KEY: test-gemini-key
    OPENAI_API_KEY: test-openai-key
  run: uv run pytest -q

build-and-push:
  name: Build And Push Image
  runs-on: ubuntu-latest
  needs:
    - test
  if: github.event_name != 'pull_request'
  permissions:
    id-token: write
    contents: read

```

```

steps:
  - name: Checkout
    uses: actions/checkout@v4

  - name: Configure AWS credentials
    uses: aws-actions/configure-aws-credentials@v4
    with:
      role-to-assume: ${{ secrets.AWS_GITHUB_ACTIONS_ROLE_ARN }}
      aws-region: ${{ env.AWS_REGION }}

  - name: Log in to ECR
    id: login-ecr
    uses: aws-actions/amazon-ecr-login@v2

  - name: Set up Docker Buildx
    uses: docker/setup-buildx-action@v3

  - name: Build and push image
    uses: docker/build-push-action@v6
    with:
      context: .
      file: ./Dockerfile
      push: true
      tags: |
        ${{ steps.login-ecr.outputs.registry }}/${{ env.ECR_REPOSITORY }}:latest
        ${{ steps.login-ecr.outputs.registry }}/${{ env.ECR_REPOSITORY }}:${{ env.IMAGE_TAG }}
      cache-from: type=gha
      cache-to: type=gha,mode=max

deploy:
  name: Deploy
  runs-on: ubuntu-latest
  needs:
    - build-and-push
  if: github.event_name != 'pull_request'

```

```

permissions:
  id-token: write
  contents: read

steps:
  - name: Configure AWS credentials
    uses: aws-actions/configure-aws-credentials@v4
    with:
      role-to-assume: ${ secrets.AWS_GITHUB_ACTIONS_ROLE_ARN }
      aws-region: ${ env.AWS_REGION }

  - name: Deploy on EC2
    env:
      SSH_HOST: ${ secrets.MEETBOWL_EC2_HOST }
      SSH_USER: ${ secrets.MEETBOWL_EC2_USER }
      SSH_KEY: ${ secrets.MEETBOWL_EC2_SSH_KEY }
      DEPLOY_PATH: ${ secrets.MEETBOWL_DEPLOY_PATH }
      AWS_REGION: ${ env.AWS_REGION }
      ECR_REPOSITORY: ${ env.ECR_REPOSITORY }
      IMAGE_TAG: ${ env.IMAGE_TAG }
    run: |
      set -euo pipefail

      mkdir -p ~/.ssh
      printf '%s\n' "$SSH_KEY" > ~/.ssh/deploy_key
      chmod 600 ~/.ssh/deploy_key
      ssh-keyscan "$SSH_HOST" >> ~/.ssh/known_hosts

      ssh -i ~/.ssh/deploy_key "$SSH_USER@$SSH_HOST" \
        "DEPLOY_PATH='$DEPLOY_PATH' AWS_REGION=ap-south
east-2 ECR_REPOSITORY='$ECR_REPOSITORY' IMAGE_TAG='$IMAGE_T
AG' bash -s" <<'EOF'
      set -euo pipefail

      cd "$DEPLOY_PATH"

      if [ ! -x ./scripts/deploy-ai.sh ]; then

```

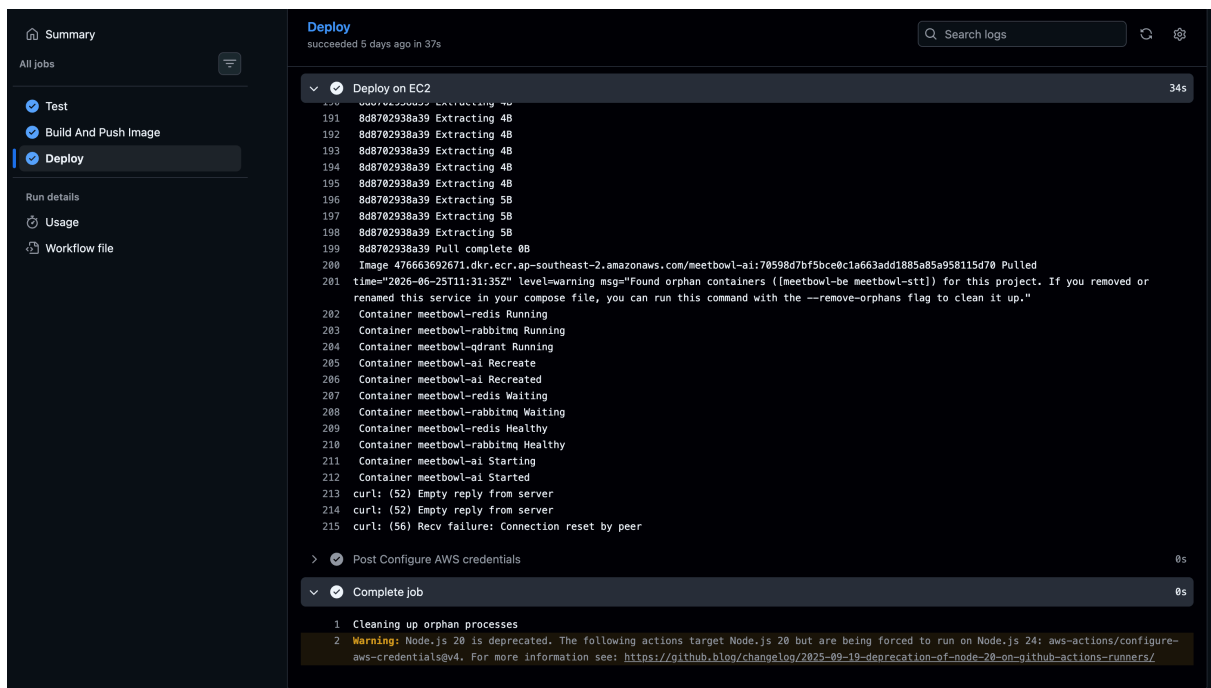
```

        echo "expected deploy script at $DEPLOY_PATH/scripts/deploy-ai.sh"
        exit 1
    fi

    ./scripts/deploy-ai.sh
EOF

```

6.4 배포 결과



7. STT Server CI/CD

STT Server는 Node.js 기반 실시간 STT 중계 서버이며, LiveKit 오디오 스트림과 외부 STT API 사이의 실시간 처리를 담당한다.

항목	내용
런타임	Node.js 22
패키지 설치	npm ci
테스트	npm test
빌드	npm run build
이미지 저장소	AWS ECR - meetbowl-stt
배포 방식	EC2 SSH 접속 후 scripts/deploy-stt.sh 실행

7.1 CI 흐름

1. Pull Request 또는 main push 발생 시 GitHub Actions가 실행된다.
2. Node.js 22 환경을 구성하고 npm 캐시를 사용한다.
3. npm ci로 의존성을 설치한다.
4. npm test로 테스트를 수행한다.
5. npm run build로 애플리케이션 빌드를 검증한다.

7.2 CD 흐름

1. 테스트와 빌드가 성공하면 Docker 이미지 빌드 job이 실행된다.
2. AWS OIDC를 통해 AWS 인증을 수행한다.
3. ECR 에 로그인한다.
4. Docker Buildx를 설정한다.
5. Docker 이미지를 빌드한다.
6. ECR에 latest 태그와 GitHub SHA 태그로 push한다.
7. deploy job에서 EC2에 SSH로 접속한다.
8. EC2 내부 infra 레포의 scripts/deploy-stt.sh를 실행한다.

7.3 배포 전략

- STT Server는 실시간 연결을 담당하므로 배포 시 기존 WebSocket 또는 LiveKit 연결이 끊길 수 있다.
- 현재 MVP 단계에서는 단일 EC2 기반 컨테이너 교체 방식으로 배포한다.
- 향후 무중단 배포가 필요할 경우 Blue-Green, Rolling Update, ECS 또는 Kubernetes 기반 구조로 확장한다.

7.4 파이프라인 코드

```
name: Meetbowl STT CI/CD

on:
  pull_request:
    branches:
      - main
```

```

push:
  branches:
    - main
workflow_dispatch:

concurrency:
  group: meetbowl-stt-${{ github.ref }}
  cancel-in-progress: true

env:
  NODE_VERSION: "22"
  AWS_REGION: ap-southeast-2
  ECR_REPOSITORY: meetbowl-stt
  IMAGE_TAG: ${ github.sha }}

jobs:
  test:
    name: Test
    runs-on: ubuntu-latest

    steps:
      - name: Checkout pull request head
        if: github.event_name == 'pull_request'
        uses: actions/checkout@v4
        with:
          ref: ${ github.event.pull_request.head.sha }}

      - name: Checkout push/workflow ref
        if: github.event_name != 'pull_request'
        uses: actions/checkout@v4

      - name: Set up Node
        uses: actions/setup-node@v4
        with:
          node-version: ${ env.NODE_VERSION }}
          cache: npm

      - name: Install dependencies

```

```

    run: npm ci

- name: Run tests
  run: npm test

- name: Build application
  run: npm run build

build-and-push:
  name: Build And Push Image
  runs-on: ubuntu-latest
  needs:
    - test
  if: github.event_name != 'pull_request'
  permissions:
    id-token: write
    contents: read

  steps:
    - name: Checkout
      uses: actions/checkout@v4

    - name: Configure AWS credentials
      uses: aws-actions/configure-aws-credentials@v4
      with:
        role-to-assume: ${{ secrets.AWS_GITHUB_ACTIONS_ROLE_ARN }}
        aws-region: ${{ env.AWS_REGION }}

    - name: Log in to ECR
      id: login-ecr
      uses: aws-actions/amazon-ecr-login@v2

    - name: Set up Docker Buildx
      uses: docker/setup-buildx-action@v3

    - name: Build and push image
      uses: docker/build-push-action@v6

```

```

    with:
      context: .
      file: ./Dockerfile
      push: true
      tags: |
        ${ steps.login-ecr.outputs.registry }/${ env
v.ECR_REPOSITORY }}:latest
        ${ steps.login-ecr.outputs.registry }/${ env
v.ECR_REPOSITORY }}:${ env.IMAGE_TAG }}
      cache-from: type=gha
      cache-to: type=gha,mode=max

deploy:
  name: Deploy
  runs-on: ubuntu-latest
  needs:
    - build-and-push
  if: github.event_name != 'pull_request'
  permissions:
    id-token: write
    contents: read

  steps:
    - name: Configure AWS credentials
      uses: aws-actions/configure-aws-credentials@v4
      with:
        role-to-assume: ${ secrets.AWS_GITHUB_ACTIONS_ROLE_ARN }
        aws-region: ${ env.AWS_REGION }

    - name: Deploy on EC2
      env:
        SSH_HOST: ${ secrets.MEETBOWL_EC2_HOST }
        SSH_USER: ${ secrets.MEETBOWL_EC2_USER }
        SSH_KEY: ${ secrets.MEETBOWL_EC2_SSH_KEY }
        DEPLOY_PATH: ${ secrets.MEETBOWL_DEPLOY_PATH }
        AWS_REGION: ${ env.AWS_REGION }
        ECR_REPOSITORY: ${ env.ECR_REPOSITORY }

```

```

IMAGE_TAG: ${ env.IMAGE_TAG }
run: |
    set -euo pipefail

    mkdir -p ~/.ssh
    printf '%s\n' "$SSH_KEY" > ~/.ssh/deploy_key
    chmod 600 ~/.ssh/deploy_key
    ssh-keyscan "$SSH_HOST" >> ~/.ssh/known_hosts

    ssh -i ~/.ssh/deploy_key "$SSH_USER@$SSH_HOST" \
        "DEPLOY_PATH='$DEPLOY_PATH' AWS_REGION='$AWS_REGION' ECR_REPOSITORY='$ECR_REPOSITORY' IMAGE_TAG='$IMAGE_TAG' bash -s" <<'EOF'
        set -euo pipefail

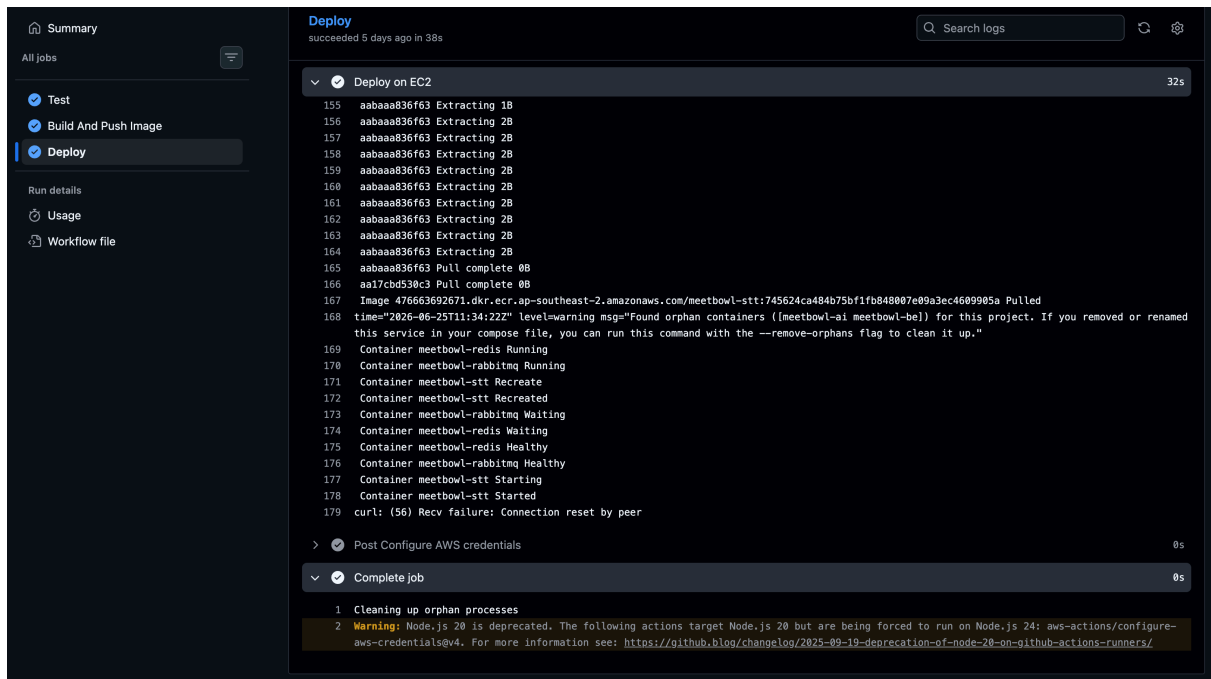
        cd "$DEPLOY_PATH"

        if [ ! -x ./scripts/deploy-stt.sh ]; then
            echo "expected deploy script at $DEPLOY_PATH/scripts/deploy-stt.sh"
            exit 1
        fi

        ./scripts/deploy-stt.sh
    EOF

```

7.4 배포 결과



8. Infra Repository 운영 방식

Infra Repository는 EC2 내부 배포 경로에 Git으로 clone되어 있으며, docker-compose.yml, 환경변수 파일, Nginx 설정, 배포 스크립트를 관리한다.

파일/디렉토리	역할
docker-compose.yml	BE, AI, STT, MariaDB, Redis, RabbitMQ, Qdrant, LiveKit, Nginx 컨테이너 실행 정의
.env 또는 서비스별 env 파일	운영 환경변수 관리
scripts/deploy-be.sh	Backend 이미지 pull 및 컨테이너 교체
scripts/deploy-ai.sh	AI Server 이미지 pull 및 컨테이너 교체
scripts/deploy-stt.sh	STT Server 이미지 pull 및 컨테이너 교체
nginx 설정	외부 요청 라우팅 및 reverse proxy 설정

8.1 배포 스크립트 역할

1. AWS ECR에 로그인한다.
2. IMAGE_TAG 기준 Docker 이미지를 pull한다.
3. docker-compose.yml 또는 env 값에 해당 서비스 이미지 태그를 반영한다.
4. docker compose up -d 명령으로 해당 서비스 컨테이너를 재생성한다.
5. docker compose ps 또는 health check로 실행 상태를 확인한다.

6. 사용하지 않는 이전 이미지를 정리한다.

9. 보안 관리

구분	관리 방식
GitHub Secrets	EC2 SSH 접속 정보, AWS Role ARN, S3 Bucket, CloudFront ID 등 저장
AWS OIDC	장기 Access Key 없이 GitHub Actions에서 IAM Role Assume
AWS SSM Parameter Store	운영 CORS 설정 등 민감하거나 변경 가능한 설정값 저장
SSH Key	GitHub Actions 실행 중 임시 파일로 생성하고 chmod 600 권한 적용
테스트 Secret	운영 Secret 대신 dummy 값 사용

- AWS Access Key와 Secret Key를 GitHub Secrets에 장기 보관하지 않는 구조를 지향한다.
- 운영 설정값은 코드에 포함하지 않고 Secret 또는 Parameter Store에서 주입한다.
- CloudFront, S3, ECR, EC2 권한은 서비스 배포에 필요한 최소 권한 원칙을 적용한다.

10. 버전 관리 및 롤백 전략

대상	버전 관리 방식	롤백 방식
Backend	ECR latest + GitHub SHA 태그	이전 정상 SHA를 IMAGE_TAG로 지정하여 deploy-be.sh 재실행
AI Server	ECR latest + GitHub SHA 태그	이전 정상 SHA를 IMAGE_TAG로 지정하여 deploy-ai.sh 재실행
STT Server	ECR latest + GitHub SHA 태그	이전 정상 SHA를 IMAGE_TAG로 지정하여 deploy-stt.sh 재실행
Frontend	GitHub SHA 기준 build artifact / S3 객체	이전 커밋 재배포 또는 S3 Versioning 복구

- latest 태그는 최신 배포 버전을 의미한다.
- GitHub SHA 태그는 특정 커밋의 불변 배포 버전을 의미한다.
- 운영 장애 발생 시 원인 분석 전에 먼저 이전 정상 SHA 이미지로 복구할 수 있다.

11. 로그 및 모니터링

- GitHub Actions 로그에서 테스트 성공 여부, Docker 이미지 빌드 결과, ECR push 결과, SSH 배포 실행 결과를 확인한다.
- EC2에서는 `docker compose ps`, `docker compose logs` 명령으로 컨테이너 상태와 서비스 로그를 확인한다.
- 배포 스크립트는 성공/실패 로그를 명확히 출력해야 한다.
- 향후 CloudWatch, Prometheus, Grafana 또는 SigNoz 기반 모니터링을 추가하여 배포 후 장애 감지를 강화한다.

12. 최종 정리

Meetbowl의 CI/CD는 서비스별 레포지토리에서 테스트와 빌드를 수행하고, Docker 이미지로 패키징한 뒤, AWS ECR과 EC2 기반 Docker Compose 배포로 운영 반영하는 구조이다. Frontend는 S3와 CloudFront를 통해 정적 파일을 배포한다. 현재 구조는 MVP 단계에서 비용과 구현 복잡도를 줄일 수 있으며, 향후 트래픽 증가와 무중단 요구가 커지면 ECS, CodeDeploy Blue-Green, 또는 Kubernetes 기반 배포 구조로 확장할 수 있다.